

Supervised Learning Assignment

General Guidelines for Assignment for Supervised Learning Component

You must submit your solutions as a Jupyter Notebook for this assignment in both .html and .ipynb formats, as well as two .csv files, blind_test_prediction.csv and blind_test_short_prediction.csv (see Task 7 for the format). The .html format will be reviewed for readability, while the .ipynb format is used to verify solutions, the csv file will be checked for accuracy. The following instructions outline the key components and guidelines for preparing the notebook:

Notebook Structure and Content

1. Headers and Subheaders:

- Organize the notebook into clearly defined sections using headers and subheaders.

2. Graphs and Visualizations:

- Include well-labeled graphs with appropriate titles, x and y-axis labels, and scales.
- Specify the colors used in each graph to ensure clarity and consistency.
- Each graph also includes a subsequent markdown cell to explain the graph's purpose and relevance to the analysis.

3. Formulas:

Include at least one formula that you actually use in your analysis (for example accuracy, a standard error or a distance) and explain in words what each part of it means. Any notation is fine: plain text, or LaTeX in a markdown cell if you prefer.

4. Lists in Markdown:

Use at least one ordered or unordered list in a markdown cell to organize information. Guidance on list formatting in Jupyter: [Here](#) is one example of a source that explains how to apply lists.

5. Work individually or in pairs:

We will contact each student to explain the work done individually or in pairs with questions to verify that the student understands the solution submitted in depth. We will only accept assignments submitted jointly by up to two students or submitted individually. The meeting will take place online (Google Meet, about 15 minutes per student). You may use AI assistants (for example ChatGPT, Claude or Copilot) to help you write code, but you are responsible for understanding every step: in the meeting we will ask how you solved the task, why you chose your methods, what your results mean and what their limitations are. We grade this understanding, not the code.

6. Grading:

Most of the grading in this section will be on evidence of applied learning in the course. Explain the reasoning behind the methods you selected. Provide visual evidence to confirm the explanations. Achieving a high level of success on performance measures may yield additional evidence of correct thought processes; however, success in performance measures is less significant in grading. Using tools you know or found online that are beyond what was taught in this course is acceptable; however, if you use such tools, please explain them at a level that a freshman majoring in engineering or hard sciences could follow as we want to follow what you are doing as well. Also, we will ask you verbally on components of your solutions so be prepared to demonstrate command of the tools you used. Knowing what your model does well, where it fails and why is as important as running it: a modest result that you understand and report honestly is worth more than a high score that you cannot explain.

7. Submission (what to upload):

- Four files, uploaded together: (1) your notebook, SupervisedLearning_<your name(s)>.ipynb, saved WITH its outputs; (2) the same notebook exported to HTML; (3) blind_test_prediction.csv (120,000 rows); (4) blind_test_short_prediction.csv (800 rows).
- Before saving and exporting, run Kernel > Restart & Run All once, so that the notebook runs from the first cell to the last and the saved outputs match the code.
- Do not upload the data files. If you worked in pairs, put both names in the first cell and submit once.
- Upload to: [the submission form](#). You need to be signed in to a Google account to upload files. Deadline: November 17, 2026. Then book your online meeting (Google Meet, November 25–26, 2026) by November 20 on [the booking page](#). If you worked in a pair, each partner books their own meeting.

The Data

The supervised learning component of the homework assignment involves analyzing a dataset structured for behavioral analysis of pairs of flies. The dataset consists of 53 Columns: the first 52 Columns represent the features extracted from the videos with the SLEAP package, while the last Column represents the labels. These labels are binary (either 0 or 1), indicating different populations of fly pairs. You are not told what the two populations are; do not assume anything about them beyond what the data show. The csv files also contain an unnamed first column with the row number; it is not a feature.

Data Structure

1. Feature Columns (1-52):

- The first 52 Columns contain features organized into 13 body parts of the flies, each with an (x, y) pixel coordinate tracked by SLEAP (Social LEAP Estimates Animal Poses).
- Each body part has separate coordinates for the male and female flies, resulting in $13 \text{ body parts} * 2 \text{ (x and y axis of coordinate)} * 2 \text{ (male and female)} = 52 \text{ feature Columns}$.

2. Label Column (53rd Column):

- The 53rd Column contains the labels, which are either 0 or 1. These labels represent different populations or behavioral states of the fly pairs.

3. **Row Chunks:**

- The data is in chunks of 150 consecutive Rows, where each chunk corresponds to 150 frames captured at 150 frames per second (fps). This structure provides 1-second snippets of video data, allowing for detailed analysis of fly behavior over short time intervals.
- Within each chunk of 150 Rows, the label (0 or 1) remains the same since all frames within a chunk are from the same pair of flies.

Dataset Composition

1. **Training Set:**

- The training set consists of 240,000 Rows, divided evenly between the two labels: 120,000 Rows with label 0 and 120,000 with label 1.
- This set trains your supervised learning models and includes the full feature set with corresponding labels.

2. **Test Set:**

- The test set contains 120,000 Rows, which are also evenly split between 60,000 Rows with label 0 and 60,000 Rows with label 1.
- This set evaluates your trained model's performance on data it has not seen before but where the labels are known.

3. **Blind Test Set:**

- The blind test set also consists of 120,000 Rows, structured identically to the test set, except no labels are provided.
- This set is for final evaluation, where you will predict the labels based on your trained model. The actual labels are withheld to ensure an unbiased assessment.

Additional Information

- **Mutual Exclusivity:** The training set is mutually exclusive from the test set and the blind test set regarding the pairs of flies they contain.
- **Repeated Pair Inclusions:** Within each set (training, test, and blind test), the same pairs of flies appear multiple times but from different parts of the video. Chunks that come from the same pair are therefore similar to each other. Keep this in mind when you split and evaluate your data (Tasks 4, 5 and 8).
- **Label Dataframes:** Two smaller, one-Column DataFrames are provided, containing the labels for the 150-Row chunks for the training and test sets. These are the "short labels" (train_short_labels.csv: 1,600 labels; test_short_labels.csv: 800 labels). Note that such a labeled DataFrame is not provided for the blind test set.
- **Link to Data:** <https://github.com/deutschlab/computational-ethology-final-project/tree/main/supervised/data> (the three large files are compressed as .csv.gz; pandas reads them directly, for example `pd.read_csv("train.csv.gz")`). Course repository with a starter notebook: <https://github.com/deutschlab/computational-ethology-final-project>

Assignment Tasks

1. Data Cleaning (4 points):

- Check for outliers or possible missing values in the dataset. (2 pts)
- Describe the identification process for outliers. (2 pts)
- Interpolation of outliers is not required.

2. Create Features (20 points):

- Create at least five features from the raw data (for example speeds, distances or angles; explain what each feature is meant to capture); at least one is temporal. (12 pts)
- Use at least two different window sizes for a temporal feature. (4 pts)
- Visually display a temporal feature with two different window sizes on the same graph. For example, plot the feature against the frame number for one 150-row chunk, with one line per window size. (4 pts)

3. Visual Testing of Features by Label (6 points):

- Make graphs to test for differences in features by labeling visually. Ensure the graphs are well-labeled and explain each graph. (2 pts)
- Which features are likely to be better predictors? Explain your answer using evidence from your graphs, and state what accuracy you expect from the models before you train them. (4 pts)

4. Train and Evaluate Models on Training Set (5 points):

- Perform an 80%/20% train-test split of the rows of the training set (a random split, for example with `train_test_split` from `scikit-learn`) and apply the following machine learning algorithms: Random Forest and one additional machine learning algorithm. (1 pts)
- Include a confusion matrix and the accuracy measure for each model, and say which type of mistake each model makes more often. (4 pts)
- If your temporal features leave some rows without a value (for example at the beginning of each window), explain what you did with those rows. Make sure that a window never runs across the boundary between two chunks.

5. Evaluation Using Full Train and Test Data (5 points):

- Apply the machine learning models and performance metrics from the previous section, but use all training and test sets data (train on the whole training set and evaluate on the test set). Display the results for comparison. (3 pts)
- Compare these results to the previous train-test split results and explain possible differences in performance. Hint: think about how similar the rows within the training set are to each other, and whether the same is true between the training set and the test set. (2 pts)

6. Enhanced Preprocessing (5 points):

- Reapply the machine learning models (for training and testing data) with (an) additional processing step(s) to structure the data to fit short labels (that is, one row and one label per 150-row chunk, for example by summarizing each feature over the chunk with its mean and standard deviation). Display the results for comparison. (5 pts)

7. Blind Test Prediction (5 points):

- Apply the enhanced preprocessing steps to the blind_test_set with the same additional preprocessing for the Random Forest model. Submit two prediction files: blind_test_short_prediction.csv (800 rows, one per chunk) and blind_test_prediction.csv (120,000 rows, one per row of blind_test.csv, from your model of Task 5). Each file must contain a column named prediction with the values 0 or 1, in the same order as the blind test data. Train your final models on the training set only (not on the test set) and state in your notebook which data each model was trained on. (5 pts)

8. Split by Chunk ([TBD] points):

- Repeat the 80%/20% split so that all 150 rows of a chunk end up on the same side of the split (for example, with GroupShuffleSplit in scikit-learn, using the chunk number as the group). Report the accuracy and compare it to the row by row split.
- Explain the difference.
- The chunk split will probably still give a higher accuracy than the test set of Task 5. Suggest at least two possible reasons.

***Why we ask:** rows from the same chunk are almost identical, so if a chunk is split between training and testing, the model is tested on data it has effectively already seen, and the accuracy is inflated.*

9. Cross-Validation ([TBD] points):

- Use 5-fold cross-validation and report the mean and standard deviation of the accuracy across the folds.
- Keep all rows of a chunk in the same fold (for example with GroupKFold).
- Is the model performance stable across folds?
- Compare the cross-validation mean with the accuracy you obtained on the test set in Task 5. Does a stable result across folds mean that the model will work equally well on the test set? Explain.

***Why we ask:** a single split gives one number that depends on luck. Cross-validation shows how much the result changes for different data, so you know how much to trust it.*

10. What is the Model Using? ([TBD] points):

- For your Random Forest, show the importance of every feature in a bar plot (for example with the feature_importances_ attribute).
- Look at the most important feature. Does it describe behavior (what the flies do), or could it reflect a physical property of the flies or of the recording (for example body size or position in the arena)? Explain.
- Retrain the Random Forest without that feature and report the accuracy on the test set. How much did it change, and what does that tell you about the model?

***Why we ask:** a model can be accurate for the wrong reason. Checking what it relies on is a first step in deciding whether to trust it.*

11. How Much Can You Trust Your Numbers? ([TBD] points):

- What accuracy would you get by guessing (the chance level) on the test set? How far above it are your models?
- The test set has 120,000 rows but only 800 chunks, and the rows of a chunk are almost identical, so the number of independent examples is closer to 800. Estimate the uncertainty of your test accuracy with the standard error $SE = \sqrt{p(1-p)/n}$, where p is the accuracy and $n = 800$. A range of about ± 2 SE contains the true accuracy with about 95% confidence.
- Are the differences between your models larger than this uncertainty? Which conclusions in your notebook are affected?

Why we ask: *two models with 71% and 72% accuracy are usually not really different. Knowing how large a difference must be to matter is part of interpreting results.*

12. Summary: Strengths and Limitations ([TBD] points):

- Write a short summary (about 6-10 sentences, in a markdown cell) that answers: (a) how well does your final model work, and how do you know; (b) where does it make mistakes (use your confusion matrix); (c) which of your evaluation numbers do you trust most and which least, and why; (d) what additional data or information would help you evaluate it better?

Why we ask: *this is the part that shows that you understand your work and not only that you ran it.*